

148 Tests Totales	126 PASS ✓	22 ERROR ✗	85% Cobertura
----------------------	---------------	---------------	------------------

1. Introducción

Este documento describe el conjunto de tests de calidad de datos implementados en el proyecto **wh_transform**, una capa de transformación dbt Core que opera sobre **Azure Synapse Analytics (Dedicated SQL Pool, dialecto T-SQL)**. El objetivo de los tests es garantizar la integridad, consistencia y corrección de los datos en todos los dominios del data warehouse.

Los tests cubren cuatro dominios productivos: **ATI Datawarehouse** (migración Korbato), **Sonnell** (pipeline de facturación), **AMA** (dashboard de flota Geotab) y **Monitoring** (salud de pipelines de ingesta). Existen dos tipos de tests: **schema tests** declarados en archivos YAML y **tests singulares** escritos en SQL.

1.1 Contexto técnico

Parámetro	Valor
Herramienta	dbt Core
Adapter	dbt-synapse (T-SQL / Azure Synapse Analytics)
Data Warehouse	ati_datawarehouse — ati-cdw-synapse.sql.azuresynapse.net
Package adicional	dbt-labs/dbt_utils (unique_combination_of_columns)
Rama del repositorio	feature/dbt-tests-data-quality
Fecha de implementación	10 de junio de 2026
Última ejecución	11 de junio de 2026 — 10:27:47 UTC

1.2 Tipos de tests en dbt

Schema tests (genéricos): declarados en archivos schema.yml. dbt genera automáticamente el SQL de validación. Tipos disponibles: *unique*, *not_null*, *accepted_values*, *relationships*, y tests de *dbt_utils* como *unique_combination_of_columns*.

Tests singulares: archivos .sql en la carpeta tests/. Contienen una query SQL que retorna las filas que *violan* la regla de negocio. Si el SELECT devuelve 0 filas → test PASA. Si devuelve filas → test FALLA con el detalle exacto.

■ Todos los tests singulares usan ref() para referenciar los modelos, garantizando que dbt resuelva correctamente el schema y el alias de cada tabla.

2. Schema Tests — ATI Datawarehouse

Archivo: models/ati_datawarehouse/schema.yml. Cubre los 7 modelos del dominio Korbato migrado al data warehouse. Los modelos son incrementales (append + DELETE pre_hook) con ventana de 1 día. Los tests validan integridad estructural: claves primarias únicas y relaciones FK entre entidades.

2.1 fleet

Materialización: table · Distribución: HASH(fleet_key) · Índice: CCI
Grain: una fila por flota (fleet_key único). Deduplicado por rn=1 en staging_fleet.

Columna	Test	Propósito
fleet_key	unique + not_null	PK — identificador único de flota. Duplicados romperían todos los JOINS downstream.
fleet_name	not_null	Nombre obligatorio. Un NULL indica ingesta incompleta del archivo fuente.
operator_id	not_null	FK lógica al operador. NULL indica registro huérfano.
_md_processed_at	not_null	Metadata de ingesta obligatoria para trazabilidad.

2.2 vehicle_day

Materialización: incremental (append) · Distribución: HASH(svc_date)
Grain: vehicle_day_key único (vehículo × día de servicio).

Columna	Test	Propósito
vehicle_day_key	unique + not_null	PK de la entidad. Duplicados causarían conteo incorrecto de días operativos.
svc_date	not_null	Fecha de servicio. Necesaria para el filtro incremental.
fleet_key	not_null + relationships → fleet	FK a fleet. Valida que cada vehículo-día pertenezca a una flota registrada.
vehicle_id	not_null	Identificador del vehículo físico.

2.3 trip

Materialización: incremental (append) · Distribución: HASH(trip_key)
Grain: trip_key único (viaje individual).

Columna	Test	Propósito
trip_key	unique + not_null	PK del viaje. Duplicados distorsionarían métricas de cantidad de viajes.
svc_date	not_null	Fecha de servicio para filtro incremental.

Columna	Test	Propósito
vehicle_day_key	not_null + relationships → vehicle_day	FK a vehicle_day. Un trip sin vehicle_day es un viaje sin vehículo asignado.
route_id	not_null	Ruta operada. NULL implica viaje sin ruta — inválido para cualquier análisis.

■ ESTADO ACTUAL: not_null en trip_key y vehicle_day_key están FALLANDO. Hay registros con NULLs en la tabla trip, originados en la fuente external.trip (data lake).

2.4 visit

Materialización: incremental (append) · Distribución: HASH(svc_date)

Grain: visit_key único (visita individual a una parada).

Columna	Test	Propósito
visit_key	unique + not_null	PK de la visita. Duplicados inflarían conteos de pasajeros.
svc_date	not_null	Fecha de servicio.
trip_key	not_null + relationships → trip	FK a trip. Una visita sin viaje es un registro huérfano.
stop_id	not_null	Parada visitada. NULL invalida análisis de OTP y APC.

■ ESTADO ACTUAL: unique en visit_key y relationships visit → trip están FALLANDO. Causa probable: visit_keys duplicados y trip_keys huérfanos derivados de los NULLs en trip.

2.5 pattern

Materialización: table · Distribución: HASH(pattern_id)

Columna	Test	Propósito
pattern_id	unique + not_null	PK del patrón de ruta. Unicidad garantiza integridad de pattern_stop.
route_id	not_null	Ruta a la que pertenece el patrón.

2.6 pattern_stop

Materialización: table · Distribución: HASH(pattern_id)

Grain: (pattern_id, stop_seq) — cada parada dentro de un patrón.

Test	Tipo	Propósito
unique_combination_of_columns (pattern_id, stop_seq)	dbt_utils	PK compuesta. Garantiza que cada posición dentro de un patrón sea única.

Test	Tipo	Propósito
pattern_id: not_null + relationships → pattern	generic	FK a pattern. Una parada sin patrón padre es un registro inválido.
stop_seq: not_null	generic	Secuencia de parada obligatoria para ordenar recorridos.
stop_id: not_null	generic	Identificador de la parada física.

2.7 Trip_Sch_Match

Materialización: incremental (append) · Distribución: ROUND_ROBIN · Índice: HEAP
Grain: (svc_date, trip_key) — matcheo de viaje real contra trip programado (GTFS).

Test	Tipo	Propósito
unique_combination_of_columns (svc_date, trip_key)	dbt_utils	PK compuesta. Un viaje solo puede matchear una vez por día.
svc_date: not_null	generic	Fecha de servicio requerida para el matcheo temporal.
trip_key: not_null + relationships → trip	generic	FK a trip. El matcheo debe apuntar a un viaje real existente.

■ ESTADO ACTUAL: relationships Trip_Sch_Match → trip está FALLANDO. Hay trip_keys en Trip_Sch_Match que no tienen registro padre en trip. Consecuencia directa de los NULLs en trip.

3. Tests Singulares — Reglas de Negocio

Los tests singulares validan lógica de negocio que los tests genéricos no pueden capturar. Cada archivo .sql en tests/ implementa una regla específica del dominio. En todos los casos: **0 filas = PASS**, **filas retornadas = FAIL** con los registros problemáticos.

Test 1: assert_sonnell_invoice_non_negative

Archivo: tests/assert_sonnell_invoice_non_negative.sql · Dominio: Sonnell · Fase: P0

Propósito

Garantiza que ninguna línea de factura de tipo **Regular** tenga un total monetario negativo. En el modelo de negocio de Sonnell, los cobros al cliente siempre son positivos. Los créditos o penalidades se registran *exclusivamente* en líneas de tipo **Offset**. Un Total < 0 en una línea Regular indica un error de cálculo en el pipeline de facturación.

Modelo validado

fct_sonnell_invoice_totals (alias: *SonnellInvoiceTotals*, schema *dbo*)

Lógica SQL

```
SELECT [Year], [Month], Subsystem, [Type], Total, Version FROM {{
  ref('fct_sonnell_invoice_totals')
}} WHERE CurrentVersion = CAST(1 AS BIT) -- solo
versión activa AND [Type] = 'Regular' -- líneas de factura normales (MB/TC/MU) AND Total
< 0 -- esto no debería existir
```

Condición de fallo

Retorna filas cuando existe algún total negativo en líneas activas de tipo Regular. El reporte de fallo incluye Year, Month, Subsystem y el Total incorrecto para facilitar la investigación.

✓ ESTADO ACTUAL: PASS — no se encontraron totales negativos en líneas Regular activas.

Test 2: assert_ama_no_future_trips

Archivo: tests/assert_ama_no_future_trips.sql · Dominio: AMA · Fase: P0

Propósito

Detecta viajes de flota AMA con actual_departure en el futuro. Un timestamp futuro en datos históricos de Geotab indica corrupción en la fuente geotab_trip, habitualmente causada por una zona horaria mal configurada en el dispositivo GPS o un error en el proceso de ingesta. Estos registros distorsionan métricas de puntualidad y cuentas de viajes del día.

Modelo validado

AMA_Geotab_Viajes (schema *dbo*, incremental con ventana de 1 día)

Lógica SQL

```
SELECT trip_id, vehicle_name, actual_departure, GETDATE() AS current_datetime,  
DATEDIFF(MINUTE, GETDATE(), actual_departure) AS minutes_ahead FROM {{  
ref('AMA_Geotab_Viajes') }} WHERE actual_departure > GETDATE()
```

Condición de fallo

Retorna filas cuando algún viaje tiene actual_departure posterior a la fecha/hora actual. El campo minutes_ahead permite cuantificar el desvío temporal.

✓ ESTADO ACTUAL: PASS — no se detectaron viajes con timestamp futuro.

Test 3: assert_sonnell_offset_cap_4_percent

Archivo: tests/assert_sonnell_offset_cap_4_percent.sql · Dominio: Sonnell · Fase: P1

Propósito

Audita la regla de negocio del **cap del 4%**: el offset (incentivo/penalidad por OTP) de cualquier subsistema no debe superar el 4% del costo total de ese subsistema en el período. Según la documentación del proyecto, esta regla es *informativa* — no bloquea la factura pero debe registrarse en SonnellOffsetApplied. Este test detecta cuando se supera el umbral para alertar al equipo de operaciones antes del cierre mensual.

Modelo validado

fct_sonnell_subsystem_offset (alias: *SonnellSubsystemOffset*)

Lógica SQL

```
SELECT [Year], [Month], Subsystem, TotalSubsystemCost, TotalOffset,  
ROUND(ABS(TotalOffset) / NULLIF(TotalSubsystemCost, 0) * 100, 2) AS offset_pct FROM {{  
ref('fct_sonnell_subsystem_offset') }} WHERE CurrentVersion = CAST(1 AS BIT) AND  
TotalSubsystemCost > 0 AND ABS(TotalOffset) / NULLIF(TotalSubsystemCost, 0) > 0.04
```

Notas técnicas

- Se usa NULLIF(TotalSubsystemCost, 0) para evitar división por cero en meses sin servicio.
- ABS(TotalOffset) cubre tanto incentivos positivos como penalidades negativas.
- El campo offset_pct en el resultado permite al equipo ver exactamente cuánto supera el 4%.

✓ ESTADO ACTUAL: PASS — ningún subsistema activo supera el cap del 4%.

Test 4: assert_sonnell_one_active_version_per_month

Archivo: tests/assert_sonnell_one_active_version_per_month.sql · Dominio: Sonnell · Fase: P1

Propósito

Valida la integridad del patrón SCD Type 2 implementado en fct_sonnell_invoice_totals. Para cada combinación de (Year, Month), debe existir exactamente **una sola versión activa** (CurrentVersion = 1). Si existen dos versiones activas para el mismo período, significa que el pre_hook de SCD Type 2 falló y no marcó la versión anterior como CurrentVersion = 0, lo que causaría que la factura se calcule doblemente.

Modelo validado

fct_sonnell_invoice_totals

Lógica SQL

```
SELECT [Year], [Month], COUNT(DISTINCT Version) AS versiones_activas FROM {{
  ref('fct_sonnell_invoice_totals') }} WHERE CurrentVersion = CAST(1 AS BIT) GROUP BY
[Year], [Month] HAVING COUNT(DISTINCT Version) > 1
```

Condición de fallo

Retorna filas con los períodos donde coexisten múltiples versiones activas. El campo versiones_activas indica cuántas versiones activas hay para ese mes.

✓ ESTADO ACTUAL: PASS — cada período tiene exactamente una versión activa.

4. Freshness de Sources — Sonnell

La configuración de **freshness** en dbt permite detectar automáticamente cuando las tablas fuente dejaron de recibir datos. A diferencia de los tests, la freshness se ejecuta con `dbt source freshness` y consulta directamente las tablas raw del data lake.

■ La freshness NO corre con 'dbt test'. Comando específico: `dbt source freshness --profiles-dir . --select source:external`

4.1 Tablas configuradas

Archivo modificado: `models/sources/external.yml`. Las tres tablas fuente del pipeline Sonnell ahora tienen freshness habilitada.

Tabla fuente	Modelo consumidor	Alias dbt	Campo	WARN	ERROR
<code>external.sonnell_trip</code>	<code>stg_sonnell_trip</code>	SonnellTrips	<code>_md_processed_at</code>	2 días	3 días
<code>external.Sonnell_checkpoint</code>	<code>stg_sonnell_checkpoints</code>	SonnellCheckpoints	<code>_md_processed_at</code>	2 días	3 días
<code>external.sonnell_subsystem</code>	<code>stg_sonnell_daily_summary</code>	SonnellDailySummary	<code>_md_processed_at</code>	2 días	3 días

4.2 Lógica de umbrales

El campo `_md_processed_at` es un DATETIME que registra cuándo el archivo fue procesado en el data lake. Si el pipeline corre diariamente, este valor tendrá típicamente entre 12 y 36 horas de antigüedad. Los umbrales se definieron así:

Escenario	Antigüedad de <code>_md_processed_at</code>	Resultado
Pipeline corrió ayer (normal)	< 36 horas	✓ PASS
Pipeline no corrió ayer	36–48 horas (aprox.)	■ WARN — dato de ayer faltante
Pipeline no corrió en 2 días	> 48 horas	■ WARN
Pipeline no corrió en 3 días	> 72 horas	✗ ERROR — escalamiento requerido

■ El umbral `warn=2 días` (en lugar de 1) existe porque dbt compara `GETDATE()` con `MAX(_md_processed_at)`. Si el pipeline corre a las 3am y se ejecuta el freshness check a las 10am, la diferencia ya es ~31 horas. Con `warn=1 día (24h)` se generarían falsos positivos en ejecuciones normales.

5. Reporte de Estado Actual — Ejecución 11/06/2026

Resultado de dbt test --profiles-dir . ejecutado el 11 de junio de 2026 a las 10:27:47 UTC. Duración total: 3 minutos 9 segundos. 6 threads en paralelo.

Métrica	Valor
Total de tests	148
Tests pasados (PASS)	126
Tests fallidos (ERROR)	22
Warnings	0
Tasa de éxito	85.1%

5.1 Errores por dominio

Dominio	Tests fallidos	Causa	Impacto	Acción
Monitoring (fct_pipeline_health_daily)	13	Incompatibilidad técnica: dbt_utils.expression_is_true no genera T-SQL válido en Synapse (Error 8155: sin alias de columna)	Falso negativo — los datos probablemente están bien, no es un problema de datos	Reemplazar con tests singulares .sql equivalentes
ATI Datawarehouse (trip, visit, Trip_Sch_Match)	6	Datos sucios en fuente external.trip: NULLs en trip_key y vehicle_day_key. Causa duplicados en visit y FK huérfanos en cascada.	Crítico — métricas de viajes, visitas y matcheo GTFS no son confiables	Investigar y limpiar external.trip en el data lake
Sonnell (fct_sonnell_subsystem_cost)	1	DailySummaryId no es único: hay registros duplicados en la tabla de costos	Medio — puede distorsionar cálculos de facturación si hay CurrentVersion=1 duplicados	Verificar si duplicados son de versiones históricas (CV=0)

Dominio	Tests fallidos	Causa	Impacto	Acción
HMS (fct_hms_monthly_trips)	1	Columna OTP tiene valores no contemplados en accepted_values ['ON_TIME', 'DELAYED']	Bajo — hay estados de viaje no mapeados en el modelo	Consultar valores reales de OTP y actualizar el test
Ejemplo dbt (my_first_dbt_model)	1	Modelo de ejemplo sin datos	Ninguno — no es productivo	Eliminar o ignorar el modelo de ejemplo

5.2 Tests fallidos — detalle completo

Test ID (resumido)	Modelo	Estado
expression_is_true: baseline_7d_stddev >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: error_count >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: error_rate_pct 0-100	fct_pipeline_health_daily	ERROR
expression_is_true: event_date <= GETDATE()	fct_pipeline_health_daily	ERROR
expression_is_true: executions_failed >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: executions_skipped >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: executions_success >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: executions_total >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: executions_total = sum partes	fct_pipeline_health_daily	ERROR
expression_is_true: expected_frequency_hrs >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: health_score 0-100	fct_pipeline_health_daily	ERROR
expression_is_true: records_failed >= 0	fct_pipeline_health_daily	ERROR
expression_is_true: records_processed >= 0	fct_pipeline_health_daily	ERROR

Test ID (resumido)	Modelo	Estado
not_null: trip_key	trip	ERROR
not_null: vehicle_day_key	trip	ERROR
unique: visit_key	visit	ERROR
relationships: trip → vehicle_day (vehicle_day_key)	trip	ERROR
relationships: visit → trip (trip_key)	visit	ERROR
relationships: Trip_Sch_Match → trip (trip_key)	Trip_Sch_Match	ERROR
unique: DailySummaryId	fct_sonnell_subsystem_cost	ERROR
accepted_values: OTP in [ON_TIME, DELAYED]	fct_hms_monthly_trips	ERROR
not_null: id (ejemplo dbt)	my_first_dbt_model	ERROR

6. Insights y Recomendaciones

6.1 Problema raíz en ATI Datawarehouse (CRÍTICO)

Los 6 errores del dominio ATI tienen un **único origen**: la tabla fuente `external.trip` en el schema `raw` contiene registros con `trip_key = NULL` y `vehicle_day_key = NULL`. El modelo `trip.sql` usa `WHERE rn = 1` para deduplicar, pero no filtra NULLs en las claves. Esto genera una cascada de fallos:

Paso	Entidad	Problema derivado
1	<code>external.trip (raw)</code>	Registros con <code>trip_key = NULL</code> ingresan al pipeline
2	<code>trip (mart)</code>	<code>not_null</code> falla — <code>trip_key</code> y <code>vehicle_day_key</code> tienen NULLs
3	<code>visit (mart)</code>	FK huérfanos — <code>visits</code> apuntan a <code>trip_keys</code> que no existen como PK válida
4	<code>visit (mart)</code>	<code>unique</code> falla — NULL en <code>trip_key</code> puede generar <code>visit_keys</code> duplicados
5	<code>Trip_Sch_Match (mart)</code>	FK huérfanos — <code>matcheo</code> apunta a <code>trip_keys</code> inválidos

Acción recomendada: agregar filtro en el modelo `trip.sql`:

```
-- En trip.sql, agregar al WHERE: AND trip_key IS NOT NULL AND vehicle_day_key IS NOT NULL
```

■ Investigar también la fuente en el data lake: ¿por qué `external.trip` tiene registros sin clave? Puede ser un problema en el proceso de ingesta o en el origen Korbato.

6.2 Incompatibilidad dbt_utils.expression_is_true con Synapse

El error `'No column name was specified for column 1 of dbt_internal_test'` (SQL Error 8155) afecta exclusivamente a los tests `dbt_utils.expression_is_true`. El macro genera SQL con una subquery derivada sin alias de columna, lo cual es inválido en SQL Server/Synapse pero válido en PostgreSQL/Snowflake.

Solución A (recomendada): reemplazar con tests singulares equivalentes. Ejemplo:

```
-- En lugar de expression_is_true: health_score >= 0 AND health_score <= 100 -- Crear tests/assert_monitoring_health_score_range.sql: SELECT event_date, entity_name, health_score FROM {{ ref('fct_pipeline_health_daily') }} WHERE health_score < 0 OR health_score > 100
```

Solución B: verificar si una versión más reciente de `dbt_utils` corrige la compatibilidad con Synapse y actualizar `packages.yml`.

6.3 DailySummaryId duplicado en Sonnell

El test `unique: DailySummaryId` falla en `fct_sonnell_subsystem_cost`. Este modelo usa `incremental_strategy='append'` con un guard `NOT EXISTS`. Los duplicados pueden ser de versiones históricas (`CurrentVersion = 0`) o, más grave, de la versión activa.

Diagnóstico recomendado:

```
-- Verificar si los duplicados son de versión activa o histórica: SELECT DailySummaryId,
COUNT(*) as cnt, MAX(CurrentVersion) as max_cv FROM dbo.dbt_SonnellSubsystemCost GROUP
BY DailySummaryId HAVING COUNT(*) > 1 ORDER BY max_cv DESC
```

■ Si `max_cv = 0` para todos los duplicados: son versiones históricas — el test puede ajustarse con `WHERE CurrentVersion = 1`. Si `max_cv = 1`: hay duplicados activos — investigar si el `pre_hook` de SCD Type 2 falló.

6.4 Valores OTP no contemplados en HMS

El test `accepted_values: OTP in ['ON_TIME', 'DELAYED']` falla en `fct_hms_monthly_trips`. La fuente `raw.hms_trips` tiene estados de viaje adicionales que no fueron mapeados en el modelo.

Diagnóstico recomendado:

```
-- Ver todos los valores reales de OTP: SELECT OTP, COUNT(*) AS cnt FROM
dbo.HMS_MonthlyDataTrip GROUP BY OTP ORDER BY cnt DESC
```

Con los valores reales, actualizar el test en `models/hms/schema.yml` para incluir todos los valores válidos, o actualizar el modelo para normalizar los estados.

6.5 Resumen de acciones por prioridad

Prioridad	Acción	Dominio	Esfuerzo
P0 — URGENTE	Filtrar NULLs en <code>trip.sql</code> (<code>trip_key IS NOT NULL</code> , <code>vehicle_day_key IS NOT NULL</code>)	ATI Datawarehouse	1h
P0 — URGENTE	Investigar origen de NULLs en <code>external.trip</code> en el data lake	ATI Datawarehouse	2-4h
P1 — IMPORTANTE	Reemplazar 13 <code>expression_is_true</code> por tests singulares T-SQL	Monitoring	3h
P1 — IMPORTANTE	Diagnosticar y corregir <code>DailySummaryId</code> duplicados en Sonnell	Sonnell	1-2h
P2 — NORMAL	Actualizar <code>accepted_values</code> de OTP con valores reales de HMS	HMS	30min
P2 — NORMAL	Eliminar o ignorar modelo de ejemplo <code>my_first_dbt_model</code>	General	15min

7. Comandos de Referencia

Correr todos los tests

```
dbt test --profiles-dir .
```

Por dominio

```
dbt test --profiles-dir . --select tag:ati_datawarehouse dbt test --profiles-dir .  
--select tag:sonnell dbt test --profiles-dir . --select tag:dashboard_AMA dbt test  
--profiles-dir . --select tag:monitoring
```

Por tipo de test

```
dbt test --profiles-dir . --select test_type:singular # solo tests .sql dbt test  
--profiles-dir . --select test_type:generic # solo schema tests
```

Modelo específico

```
dbt test --profiles-dir . --select fleet dbt test --profiles-dir . --select  
fct_sonnell_invoice_totals dbt test --profiles-dir . --select AMA_Geotab_Viajes
```

Freshness (comando separado)

```
dbt source freshness --profiles-dir . --select source:external # Sonnell (3 tablas) dbt  
source freshness --profiles-dir . # todas las fuentes
```

Documento generado automáticamente por el equipo de Data Engineering — wh_transform v1.0
11 de junio de 2026 · Azure Synapse Analytics · dbt Core